

Principe

Dans ce tuto on utilise une manette de jeu

Ainsi le navigateur expose la manette via une fonctionnalité : Gamepad API

Le code, à chaque frame, interroge le navigateur pour avoir l'état du gamepad, puis applique sa logique : logique de jeu, mise à jour physique et visuelle.

La capture

```
// ./controle/gamepad.js
import { floor } from '../tool/function'

const ATTACK = 0

const JUMP = 1

const LOCK = 7

const X = 0

const Z = 1

export default class Gamepad {

  get gamepad() {

    return navigator.getGamepads()[0]

  }

  get x() {

    if(!this.gamepad) return 0

    return floor(this.gamepad.axes[X])

  }

  get z() {
```

```

    if(!this.gamepad) return 0

    return floor(this.gamepad.axes[Z])
}

get attack() {

    if(!this.gamepad) return false

    return this.gamepad.axes[ATTACK].pressed
    // erreur de code?: return this.gamepad.buttons[ATTACK]?.pressed

}

get jump() {

    if(!this.gamepad) return false

    return this.gamepad.axes[JUMP].pressed

}

get lock() {

    if(!this.gamepad) return false

    return this.gamepad.axes[LOCK].pressed

}
}

```

La fonction floor est un methode pour définir une zone morte entre les axes x/z analogiques

```

// ./tool/function.js
export function floor(float, max = 0.2) {

    return Math.abs(float) < max ? 0 : float

}

```

Ainsi on applique la logique dans l'entité player

```
import Gamepad from "../control/gamepad";

const SPEED = 3

export default class Player extends Object3D {

  collider = null

  rigidBody = null

  ctrl = new Gamepad()

  ...

  updatePhysic() {

    const x = this.ctrl.x * SPEED

    const z = this.ctrl.z * SPEED

    const y = this.rigidBody.linvel().y

    this.rigidBody.setLinvel({x,y,z}, true)

  }

  ...
}
```